Data Science Assignment: Cost-Sensitive Customer Churn Prediction

Instructor: Dr. Kheradpisheh
Department of Computer Science
Shahid Beheshti University — Spring 2026

Introduction

In this assignment, you will engineer an end-to-end predictive analytics pipeline using a real-world telecommunications dataset to forecast customer attrition (churn). This project replicates the comprehensive workflow of an enterprise data science initiative: data ingestion and quality auditing, exploratory diagnostic analysis, feature synthesis, robust class-imbalance mitigation, multi-model classification, hyperparameter optimization, and post-modeling interpretability.

Crucially, this assignment moves beyond traditional, symmetric statistical metrics (such as raw accuracy) to introduce cost-sensitive evaluation frameworks. You will optimize decision thresholds against an asymmetric business utility function that balances the financial trade-offs of customer retention. Finally, your optimal model will be deployed to generate test predictions formatted for automated evaluation via a judge system.

Objectives

The primary objective is to bridge the gap between statistical modeling and tangible business utility. By completing this assignment, you will demonstrate proficiency in:

- **Exploratory & Diagnostic Analysis:** Uncovering latent statistical dependencies and non-linear interactions through programmatic visual and numerical diagnostics.
- **Data Remediation:** Systematically handling data type anomalies, implicit missingness, and stark target class skews.
- **Domain-Driven Feature Engineering:** Translating customer behavioral hypotheses into predictive mathematical representations.
- **Comparative Model Evaluation:** Implementing, cross-validating, and benchmarking nine distinct statistical learning architectures.
- **Cost-Sensitive Optimization:** Constructing empirical decision-threshold curves to maximize an explicit business profit function.
- **Model Explainability:** Leveraging game-theoretic or permutation-based feature attribution methods to extract actionable operational insights.

Problem Statement

Customer attrition directly threatens subscription-based business models. A telecommunications corporation seeks to mitigate revenue loss by proactively identifying subscribers with a high propensity to terminate their contracts.

Your goal is to construct a highly calibrating classification model $f(X) \to Y \in \{0, 1\}$ utilizing customer demographics, subscribed service profiles, and account financial telemetry. Your downstream objective is to isolate high-risk customers so that a targeted, cost-constrained marketing retention campaign can be deployed efficiently.

Dataset Description

The assignment utilizes the standardized Telco Customer Churn benchmark dataset, comprised of 7,043 distinct subscriber records. The data schema spans 20 feature covariates and a single binary target indicator, `Churn` (Yes $= 1$, No $= 0$), which measures whether a customer severed their contract within the trailing 30-day window. The feature space is divided into three functional dimensions:

- **Demographic Attributes**: `gender`, `SeniorCitizen` (binary), `Partner`, `Dependents`.
- **Service Architecture**: `PhoneService`, `MultipleLines`, and web provisions including `InternetService`, `OnlineSecurity`, `OnlineBackup`, `DeviceProtection`, `TechSupport`, `StreamingTV`, `StreamingMovies`.

- **Account Financials**: `tenure` (measured in months), `Contract` type, `PaperlessBilling`, `PaymentMethod`, `MonthlyCharges`, and `TotalCharges`.

Data Quality Note: The `TotalCharges` covariate contains structural anomalies, including unmasked blank strings that coerce the column into a generic text data type. You must programmatically isolate, parse, and remediate these malformed entries during your data preparation phase.

Tasks

1. Exploratory Data Analysis and Data Preparation

Conduct a rigorous data audit to diagnose structural characteristics and underlying distributions before fitting any models.

- **Structural Inspection:** Programmatically verify dimensions, schema data types, and null configurations. Generate descriptive statistical summaries (mean, variance, quantiles, and category cardinalities).
- **Data Cleaning & Strategic Imputation:** Isolate the blank strings within `TotalCharges`, cast the feature to a floating-point numeric type, and apply a conditional imputation strategy (e.g., replacing `NaN` values with the median value conditional on the customer's `tenure` cohort). **Justify your chosen design pattern.**
- **Univariate Architecture:** Plot empirical distributions for all covariates. Map categorical attributes using balanced bar plots and continuous metrics via kernel density estimations (KDE) or normalized histograms. Quantify the exact target class imbalance ratio.
- **Bivariate Dissection & Correlation:** Analyze the statistical association between predictors and the target variable. Visualize continuous metrics against churn via split violin or box plots. Compute a comprehensive Pearson/Spearman correlation matrix for numerical elements, and map it as a annotated heatmap.
- **Analytical Synthesis:** Document your analytical deductions. Which demographic profiles or contract frameworks exhibit the highest empirical probability of churn? Identify potential collinearities or highly discriminative features.

2. Feature Engineering

Synthesize ten custom features designed to encapsulate operational behavioral hypotheses. For every feature engineered, write a concise analytical hypothesis explaining the business intuition and why it yields predictive signal. Implement the following:

- `TenureRange`: An ordinal, binned representation of continuous tenure: $[0, 12]$, $(12, 24]$, $(24, 36]$, $(36, 48]$, $(48, 60]$, $(60, \infty)$ months.
- `TotalServices`: An aggregate discrete metric calculating the total number of core services active for a customer out of the 8 available service components. Non-service responses (e.g., "No internet service") must be numerically zeroed.
- `Security`: A boolean interaction flag evaluating to 1 if the account subscribes to *both* `OnlineSecurity` and `DeviceProtection`, else 0.
- `Entertainment`: A boolean interaction flag evaluating to 1 if the account subscribes to *both* `StreamingTV` and `StreamingMovies`, else 0.
- `SeniorTechSupport`: An interaction feature mapping vulnerable or high-reliance segments; equals 1 if `SeniorCitizen` $= 1$ and `TechSupport` $=$ "Yes", else 0.
- `BillingAndPayment`: A high-friction operational flag evaluating to 1 if `PaperlessBilling` $=$ "Yes" and `PaymentMethod` $=$ "Electronic check", else 0.

- `MonthlyChargesRange`: A discretized, fixed-width representation of `MonthlyCharges` utilizing bin widths of 20 units.
- `AvgMonthlyPerService`: A continuous efficiency ratio computed as $\frac{\texttt{MonthlyCharges}}{\texttt{TotalServices}}$. Handle edge cases where $\texttt{TotalServices} = 0$ by mapping to the global median of `MonthlyCharges`.
- `TenureContractRisk`: A structural volatility indicator evaluating to 1 if a customer maintains a month-to-month contract while concurrently having a $\texttt{tenure} \leq 12$ months, else 0.
- `LoyaltyFactor`: A custom composite ordinal score ($\in [1, 5]$) mapped systematically from a joint matrix of contract commitment and historical tenure duration. Explicitly define and defend your scoring hierarchy.

Following feature generation, vectorize your non-numeric components. Apply binary label encoding to strict dual-state variables and one-hot encoding to multi-categorical attributes. **To preserve strict evaluation hygiene and eliminate data leakage, embed all transformers and vectorizers within a scikit-learn `Pipeline` framework, or ensure that transformations learn parameters solely from the training partitions.**

3. Class Imbalance Mitigation

The target variable exhibits a pronounced class asymmetry that risks biasing classifiers toward high-precision, zero-recall majority class predictions. You must explicitly address this behavior:

- **Baseline Assessment:** Establish an unweighted baseline by fitting a default Random Forest classifier on the native, raw training partition. Report the resulting validation confusion matrix, highlighting the baseline minority-class recall.
- **Synthetic Oversampling:** Introduce a synthetic oversampling methodology (e.g., SMOTE or ADASYN) exclusively within your training folds. Re-evaluate the baseline model to assess the shifts in minority recall and F1-score.
- **Cost-Penalized Learning:** Evaluate algorithmic class weighting by adjusting the `class_weight` parameter to penalize false negatives inversely proportional to class frequencies.
- **Comparative Review:** Compile a structured benchmarking table contrasting your imbalance remediation strategies against your unweighted baseline. Detail the mathematical trade-offs observed between precision and recall.

4. Model Implementation and Hyperparameter Tuning

Implement, validate, and tune nine distinct classification paradigms. For the estimators marked with an asterisk (*), you are required to perform systematic hyperparameter optimization using the prescribed cross-validated search mechanism. For the remaining models, you may use robust default settings or perform discretionary tuning.

1. Decision Tree Classifier
2. Naive Bayes (GaussianNB)
3. AdaBoost Classifier
4. Multilayer Perceptron (MLP)
5. Bagging Classifier (utilizing a Random Forest base estimator)
6. K-Nearest Neighbors (KNN) *via GridSearchCV **
7. Logistic Regression *via GridSearchCV **
8. Random Forest *via RandomizedSearchCV **
9. Support Vector Machine (SVM) *via GridSearchCV **

Execution requirements for each model pipeline:

- Partition the structured dataset using an 80/20 stratified split, ensuring that the empirical target distribution is exactly preserved across both training and validation sets.
- For targeted estimators (*), specify a wide, well-reasoned hyperparameter search grid (e.g., regularization penalty parameters C and kernel functions for SVM; $n\_estimators$ and structural max_depth limits for tree ensembles). Conduct search operations via stratified 5-fold cross-validation bounded strictly to the training fold.
- Extract the optimal parameter configuration and re-train the model on the full training partition.
- Generate validation predictions and extract complete classification reports containing accuracy, precision, recall, macro F1-scores, and explicit confusion matrices. Save the continuous predicted class probabilities, $P(Y = 1|X)$, for downstream evaluation. For the SVM architecture, explicitly enable probability estimation.

5. Validation, Benchmarking, and Business Utility Optimization

Consolidate your modeling results by constructing a comprehensive validation matrix containing accuracy, precision, recall, and F1-scores across all nine model instances.

Next, pivot from purely statistical metrics to an empirical business utility framework. Assume an asymmetric cost-benefit structure where successfully identifying an active churner allows the firm to deploy a targeted retention action, saving subscriber revenue that would otherwise be permanently lost. Conversely, misclassifying a loyal customer incurs an unnecessary operational marketing cost.

Let a True Positive (TP) yield a financial utility of $+100$ monetary units, and let a False Positive (FP) incur a financial penalty of -20 units. The empirical Net Profit ($\mathcal{U}$) equation is defined as:

$$\mathcal{U} = 100 \cdot TP - 20 \cdot FP$$

Using the validation set probabilities $P(Y = 1|X)$ generated by your classifiers, systematically sweep the decision threshold τ across the continuous spectrum $[0, 1]$ in increments of 0.01. For each incremental threshold:

1. Convert raw continuous probabilities into discrete classifications: $\hat{Y} = \mathbb{I}(P(Y = 1|X) \geq \tau)$.
2. Compute the resultant validation TP and FP frequencies.
3. Calculate the empirical Net Profit $\mathcal{U}(\tau)$.

Identify the exact optimal threshold τ^* that maximizes net financial utility for each capable model. Append two new columns to your primary performance matrix: **Maximum Net Profit** and **Optimal Threshold (τ^*)**.

Conclude this section with a thorough analytical discussion: Which estimator yields the highest financial utility? Does the top-performing profit model map directly to the model with the highest raw accuracy or F1-score? **Formally evaluate the underlying data and mathematical reasons why an estimator with moderate traditional metrics can outperform others within an asymmetric cost framework.**

6. Model Interpretability & Feature Attribution

Isolate the single optimal model that maximizes the business utility function. Transition this model out of the "black box" paradigm by applying an advanced model-agnostic feature attribution methodology: either SHAP (SHapley Additive exPlanations) or Permutation Feature Importance.

Generate a high-resolution summary visualization (e.g., a SHAP beeswarm plot or a sorted permutation importance distribution chart). Provide an authoritative analytical narrative interpreting the top five features driving your model's predictions. Connect these post-hoc feature importances directly to the empirical trends uncovered during your initial exploratory data analysis.

7. Test Prediction Submission Workflow

Deploy your optimal architecture to generate test predictions for submission. You may utilize your single highest-utility model, construct a optimized ensemble blend, or re-train your top architecture across the unified dataset using the empirically determined optimal threshold τ^*.

- **Data Hygiene Guardrail:** Ensure that your entire workflow isolates a clean, un-entangled evaluation set. If utilizing an unlabelled external evaluation partition provided for the autograder, run your pipeline cleanly over this data without leakage.
- **File Formatting:** Structure your output file into a clean CSV format containing exactly two columns: `id` (the unique customer identifier) and `label` (the binarized classification output, where No Churn $=$ 0 and Churn $= 1$).
- Verify that the output is named exactly `predictions.csv` and upload the artifact to the Quera judge platform adhering strictly to the submission parameters outlined in the course portal.

Deliverables

All deployment components must be compressed into a single archive structured as: `YourGroupName_Project.zip`. The archive must contain:

1. **Technical Research Report** (PDF format, strictly bounded to a maximum of 15 pages) containing:
   - Executive summary, formal introduction, and problem formulation.
   - Detailed EDA diagnostics, accompanied by key annotated visualizations and data cleaning declarations.
   - Structural inventory of engineered features alongside their respective behavioral hypotheses.
   - Methodological breakdown of class imbalance experiments and comparative benchmarks.
   - Hyperparameter search space matrices and cross-validation search diagnostics.
   - Core model evaluation tables detailing traditional statistical metrics alongside validation confusion matrices.
   - Financial utility curve analysis, optimal threshold mapping (τ^*), and business trade-off analysis.
   - Model explainability outputs paired with corporate-level insights.
   - Rigorous technical conclusion and programmatic pathways for model iteration.
2. **Source Code Inventory**: Well-commented, fully reproducible, and executable Jupyter Notebook(s) documenting the complete workflow from raw data ingestion to final prediction synthesis. **All random state parameters across cross-validation splits, sampling techniques, and estimators must be hardcoded to a static seed to ensure complete reproducibility.**
3. **Prediction Output**: The final, validated `predictions.csv` submission file, verifying structural compliance with the online judge requirements.

Technical Constraints & Guidelines

- Execution must be based in Python, leveraging core ecosystem libraries: `pandas`, `numpy`, `matplotlib`, `seaborn`, and `scikit-learn`. Advanced auxiliary packages such as `imbalanced-learn` (for sampling) and `shap` (for attribution) are highly encouraged.
- Arbitrary or stochastic hyperparameter adjustments are prohibited; optimization workflows must rely exclusively on cross-validated `GridSearchCV` or `RandomizedSearchCV`.
- Students are permitted to implement architectures beyond the 9 required variants (e.g., XGBoost, LightGBM, or custom stacking classifiers) provided that the required core estimators are completely mapped. Demonstrating optimized performance improvements through these advanced models will be factored directly into competitive bonus point scoring.

Grading Criteria & Rubric

The assignment is evaluated out of a total of **150 regular points**, with an additional **20 bonus points** available for exceptional implementations.

1. Core Machine Learning Pipeline [100 Points Total]

- **Exploratory Data Analysis & Data Ingestion (20 Points):** Programmatic execution of structural diagnostics, variable type parsing, and robust conditional missingness remediation (such as the `TotalCharges` conversion and tenure-based cohort imputation).
- **Feature Synthesis & Pipeline Hygiene (25 Points):** Proper categorical vectorization, explicit design patterns to prevent data leakage (using scikit-learn `Pipeline` frameworks), and providing a sound empirical rationale for each of the 10 custom variables.
- **Class Imbalance Mitigation (15 Points):** Systematic implementation and evaluation of unweighted baselines, algorithmic cost weights, and synthetic oversampling methods (such as SMOTE or ADASYN).
- **Multi-Model Evaluation & Optimization (40 Points):** Correct structural 80/20 stratified partitioning, alongside systematic, stratified 5-fold cross-validated grid sweeps (`GridSearchCV` or `RandomizedSearchCV`) across the 9 specified architectures.

2. Business Utility & Explainable AI [50 Points Total]

- **Cost-Sensitive Threshold Optimization (25 Points):** Programmatic implementation of the iterative probability sweep ($\tau \in [0, 1]$ in steps of 0.01) to map the empirical validation curve and locate the exact threshold (τ^*) that maximizes net financial profit ($\mathcal{U}$).
- **Post-Hoc Feature Attribution (15 Points):** Successful execution of advanced model-agnostic interpretation frameworks (such as game-theoretic SHAP values or Permutation Feature Importance) to decode the optimal model's internal decision matrix.
- **Analytical & Operational Narrative (10 Points):** Writing an authoritative business summary that bridges your post-hoc feature attribution findings directly back to early exploratory data data trends.

3. Competitive Bonus Framework [Up to +20 Points]

- **Advanced Modeling (+10 Points):** Discretionary deployment of modern gradient boosting frameworks (such as LightGBM, or CatBoost) and ensemble stacking layers that extend meaningfully beyond the core 9 baseline configurations.
- **Leaderboard Standing (+10 Points):** Final placement ranking achieved relative to the leaderboard.

Mandatory Deductions & Penalties

- **Stochastic Non-Reproducibility (-10 Points):** Failure to declare fixed global random state seeds across dataset samplers, validation cross-validation splits, or randomized estimators.
- **Academic Integrity & Fraud Violation (Zero Grade / Disciplinary Action):** Fabricating data, falsifying evaluation metrics, or presenting a willful discrepancy between your automated online judge leaderboard submissions and the actual, reproducible accuracy/profit generated within your submitted Jupyter Notebook.
- **Data Leakage Faults (-30 Points):** Global execution of scaling, categorical vectorization, or synthetic oversampling on the unified dataset prior to the strict structural isolation of internal training and validation folds.
- **Report Page Budget Deficit (-10 Points):** Generating a project report that exceeds the strict maximum limit of 15 pages.

Stay hungry, stay foolish.